

REPÚBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

**UNIVERSIDAD NACIONAL EXPERIMENTAL DE TELECOMUNICACIONES E
INFORMATICA (UNETI)**

PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Informe técnico – “App-práctica Ionic – Dev-Portafolio, Mejoras”

Ejercicio práctico 2 Mejoras y Actualizaciones basadas en Ejercicio 1

Autor:

Docente: MSc Carlos Márquez

C.I.: V-12.507.695 Leonardo González González

Caracas, junio de 2026

Índice

INTRODUCCIÓN	3
ANÁLISIS Y ARQUITECTURA	4
MEJORAS ÁREAS POTENCIALES	11
REPOSITORIO DE CÓDIGO	10
CONCLUSIONES	11
REFERENCIAS	12

Introducción

En el panorama tecnológico actual, las aplicaciones móviles se han convertido en la principal herramienta de interacción entre las empresas y los usuarios. Sin embargo, desarrollar una aplicación distinta para cada tipo de teléfono (Android y iOS) suele ser costoso y requiere mucho tiempo. Para resolver este problema, la industria del software ha adoptado el desarrollo "híbrido" o multiplataforma, el cual permite escribir el código una sola vez y ejecutarlo en cualquier dispositivo.

El presente proyecto académico consiste en el diseño y desarrollo de una aplicación móvil interactiva utilizando Ionic Framework en conjunto con Angular. El objetivo principal es construir una Single Page Application (Aplicación de Página Única) que funcione como un portafolio profesional e integre tres vistas principales: Inicio, Información Personal y Contacto, todas gestionadas a través de un Menú Lateral (Side Menu).

Más allá de cumplir con un requerimiento visual, el proyecto se construyó aplicando estándares modernos de la industria del software. Se implementó la arquitectura de Standalone Components (Componentes Independientes), eliminando la dependencia de módulos tradicionales para optimizar el rendimiento y el tiempo de carga de la aplicación. Esta herramienta no solo demuestra habilidades de programación estructurada, sino que también sirve como una vitrina tecnológica que fusiona el análisis de procesos administrativos con el desarrollo de soluciones digitales eficientes.

Análisis y Arquitectura de ejercicio práctico nro. 1 - denominado para lo sucesivo como:
“app-académica – Dev Portafolio”

La app-académica-Dev Portafolio es una aplicación diseñada con la combinación de los frameworks Ionic y Angular, fusionados para desarrollar aplicaciones de alto rendimiento, escalables y diseño consistente en Android y iOS, por lo que garantiza ser adaptables a cualquier dispositivo, se aplicaron los últimos estándares de la industria, tales como:

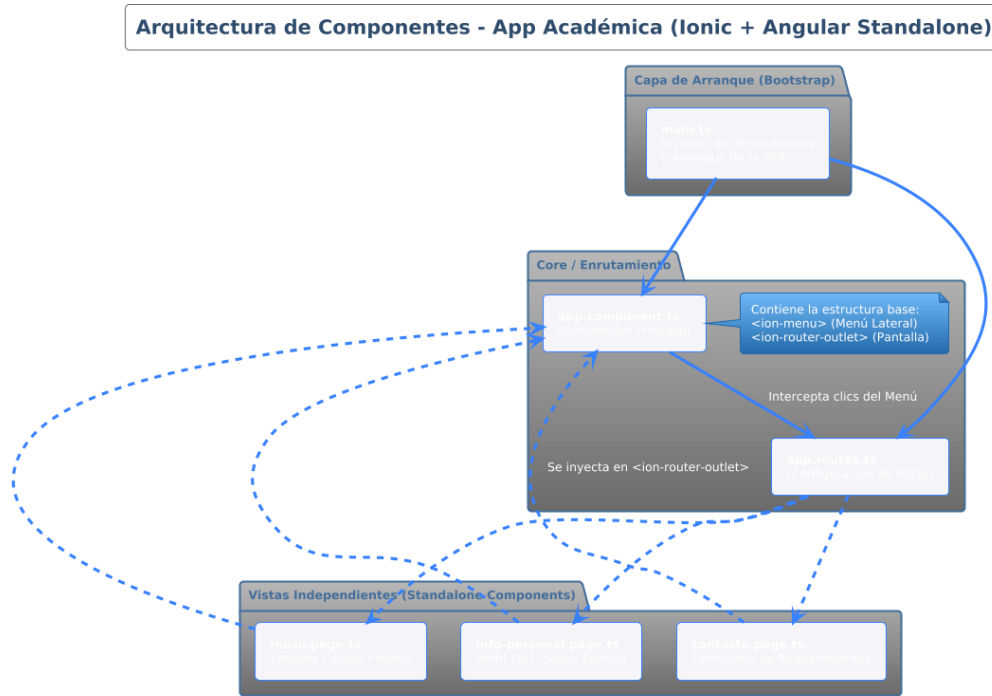
- **Standalone Components:** Cero dependencias de NgModules. Cada página y componente es una isla independiente que importa estrictamente lo que necesita, mejorando el *Tree Shaking* y el tiempo de carga.
- **Enrutamiento Funcional:** Uso de `loadComponent` en `app.routes.ts` para implementar *Lazy Loading* (carga perezosa) nativo y optimizado.
- **Control Flow Moderno:** Implementación de la nueva sintaxis `@for` y `@if` de Angular 17+ directamente en los templates HTML, dejando atrás directivas estructurales pesadas como `*ngFor`.
- **UI/UX (Mobile First):** Interfaz construida con Ionic Framework, implementando el patrón *Side Menu* (Menú Lateral), grillas responsivas y diseño adaptativo.

Desarrollo inicial del Proyecto

Se configura el ambiente de desarrollo, en mi caso estoy usando WSL2 Ubuntu, ya previamente preparado por ejercicios anteriores, solo instale **\$ npm i -g @ionic/cli**, seguido de eso cree la carpeta del Proyecto con el comando:

\$ ionic start app-academica sidemenu --type=angular

Arquitectura General:



Estructura de Navegación

La aplicación cuenta con 3 módulos principales:

- **Inicio (/inicio):** Landing page tipo "Hero" que expone la visión de marca, integrando conceptos Fintech y soluciones de infraestructura IT.
- **Información Personal (/info-personal):** Un currículum interactivo que detalla el Stack Tecnológico (MERN, Angular, Linux) y la formación académica usando componentes visuales avanzados (Badges, Avatars, Cards).
- **Contacto (/contacto):** Vista de interacciones que incluye información de localización y un formulario UI limpio para la captación de requerimientos.

Estructura del ejercicio:

```

leonardo@DESKTOP-D4RBSC6: ~/Repositorios/Univ
leonardo@DESKTOP-D4RBSC6:~/Repositorio
├── app.component.html
├── app.component.scss
├── app.component.spec.ts
├── app.component.ts
├── app.routes.ts
├── contacto
│   ├── contacto.page.html
│   ├── contacto.page.scss
│   ├── contacto.page.spec.ts
│   └── contacto.page.ts
├── folder
│   ├── folder.page.html
│   ├── folder.page.scss
│   ├── folder.page.spec.ts
│   └── folder.page.ts
├── info-personal
│   ├── info-personal.page.html
│   ├── info-personal.page.scss
│   ├── info-personal.page.spec.ts
│   └── info-personal.page.ts
└── inicio
    ├── inicio.page.html
    ├── inicio.page.scss
    ├── inicio.page.spec.ts
    └── inicio.page.ts

```

Con estos comandos, cree las tres páginas solicitadas para diseñar la estructura requerida del ejercicio práctico.

\$ ionic generate page inicio

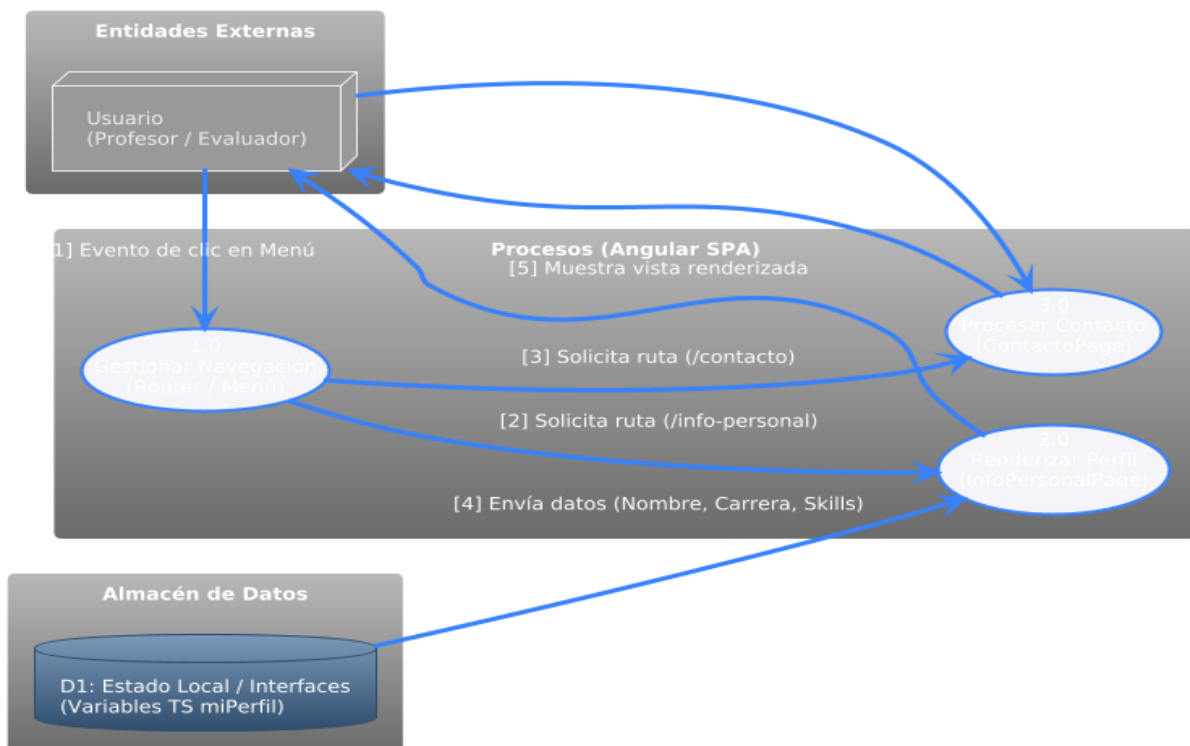
\$ ionic generate page info-personal

\$ ionic generate page contacto

Componentes e Interfaces:

- **Standalone Components (El fin de los NgModules):** Modelo de las versiones actuales de Angular 17+ lo que mejora el tree shaking, proceso donde el compilador elimina el código muerto que no usa.
- **El nuevo Control Flow (@for):** Dejar atrás la vieja directiva *ngFor y usar la sintaxis moderna nativa. Es más rápida, más limpia de leer y demuestra que no hiciste un "copiar y pegar" de un tutorial viejo del 2019.
- **Navegación SPA:** Aplicación del duo <ion-menu> + <ion-router-outlet>, patrón de las Single Page Applications (SPA). Esto logra un elegante menú lateral estático mientras el contenido central cambia dinámicamente, sin que la pantalla parpadee o recargue el navegador.

Diagrama de Flujo de Datos:



A continuación, adjuntaré las pantallas en orden desde el comienzo:

Crear la App-académica con:

ionic start app-academica sidemenu --type=angular

Ingreso a la carpeta:

```
leonardo@DESKTOP-D4RBSC6: ~/Repositorios/Universidad/ionic$ ls
app-academica
leonardo@DESKTOP-D4RBSC6:~/Repositorios/Universidad/ionic$
```

```
leonardo@DESKTOP-D4RBSC6: ~/Repositorios/Universidad/ionic/app-academica$ ls
README.md      capacitor.config.ts  karma.conf.js  package-lock.json  src              tsconfig.json
angular.json    ionic.config.json   node_modules  package.json       tsconfig.app.json  tsconfig.spec.json
leonardo@DESKTOP-D4RBSC6:~/Repositorios/Universidad/ionic/app-academica$
```

Compilar y Levantar el servidor:

```
leonardo@DESKTOP-D4RBSC6: ~/Repositorios/Universidad/ionic/app-academica$ npm run build
> app-academica@0.0.1 build
> ng build

Initial chunk files | Names | Raw size | Estimated transfer size
chunk-6FR06M07.js | - | 628.38 kB | 123.50 kB
styles-D15BGN4B.css | - | 46.30 kB | 5.43 kB
chunk-IATTSY0X.js | - | 36.28 kB | 12.62 kB
polyfills-T0CRLVKL.js | polyfills | 34.05 kB | 11.33 kB
chunk-T44ZSS69.js | - | 10.16 kB | 2.63 kB
chunk-5R7G2T4E.js | - | 9.03 kB | 3.34 kB
chunk-YJ5TA4PT.js | - | 5.74 kB | 2.13 kB
main-50A0FZW7.js | main | 5.70 kB | 1.57 kB
chunk-2Y0AKI12H.js | - | 2.50 kB | 1.00 kB
chunk-2N1B0UAM.js | - | 998 bytes | 998 bytes
chunk-TI1S22R1.js | - | 971 bytes | 971 bytes
chunk-4UHGWF85.js | - | 932 bytes | 932 bytes
chunk-Y4Q7PM72.js | - | 833 bytes | 833 bytes
chunk-4CLCTA77.js | - | 830 bytes | 830 bytes
chunk-1S47M0KC.js | - | 544 bytes | 544 bytes
chunk-YUNOCBLG.js | - | 103 bytes | 103 bytes
chunk-NMF17510.js | - | 99 bytes | 99 bytes
| Initial total | 784.05 kB | 168.87 kB

Lazy chunk files | Names | Raw size | Estimated transfer size
chunk-VHERG1IP.js | p-Cuv-vmkN | 5.00 kB | 1.99 kB
chunk-EIDSMSQ.js | info-personal-page | 4.88 kB | 1.74 kB
chunk-IVCUBRRE.js | contacto-page | 3.41 kB | 1.31 kB
chunk-UR2HQK4V.js | inicio-page | 3.10 kB | 1.28 kB
chunk-UK18RRIY.js | p-BgvIQM46 | 1.02 kB | 757 bytes
chunk-CASEXNPO.js | p-BmVRXR1y | 948 bytes | 948 bytes
chunk-LLVGLZZU.js | p-C25nLPGT | 710 bytes | 710 bytes
chunk-4U3VT15G.js | p-CneGxKsZ | 524 bytes | 524 bytes
chunk-PACG08S.js | p-D0Vn7Zh | 404 bytes | 404 bytes
chunk-X502PG76.js | p-VebVo2hO | 286 bytes | 286 bytes
chunk-VDULXYHM.js | p-CBzELu-H | 226 bytes | 226 bytes
chunk-2REB0R6E.js | p-CU1SSH8 | 211 bytes | 211 bytes
chunk-YX0AFFXD.js | p-CL0B-RWE | 127 bytes | 127 bytes

Application bundle generation complete. [18.307 seconds] - 2026-05-08T00:36:32.359Z

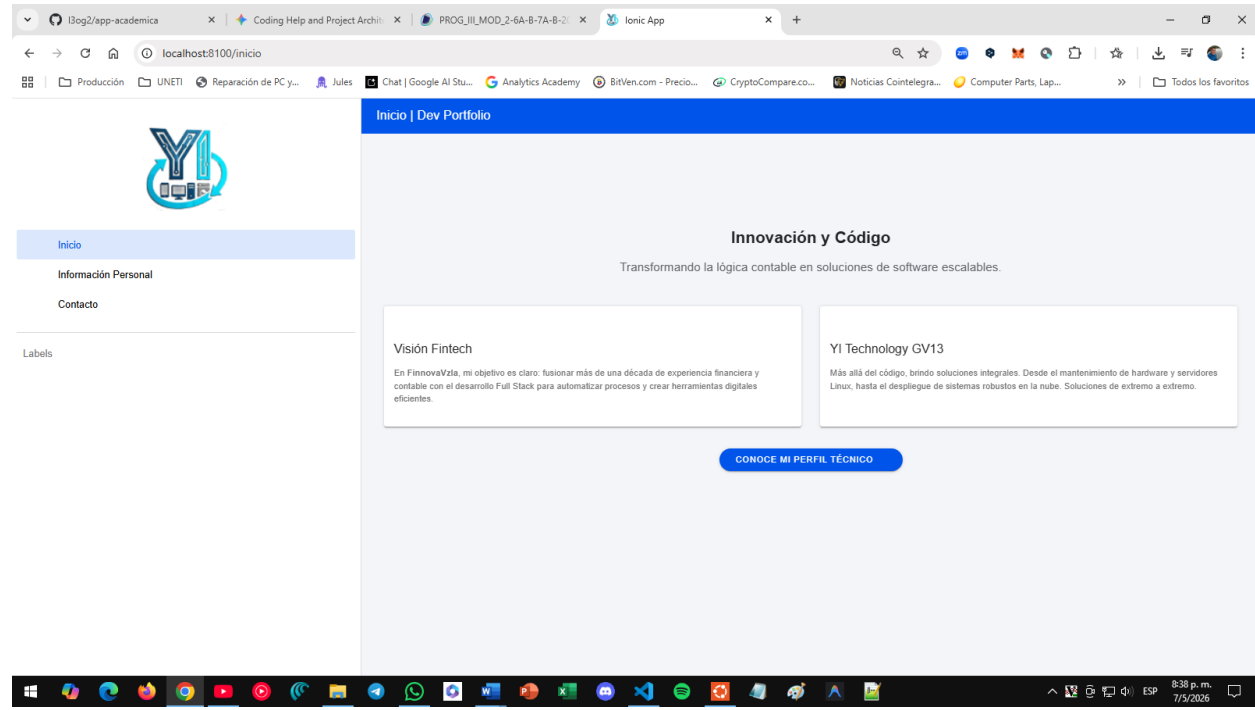
WARNING NG8113: IonNote is not used within the template of AppComponent [plugin angular-compiler]
src/app/app.component.ts:12:109
12 | ...nt, IonList, IonListHeader, IonNote, IonMenuToggle, IonItem, Io...
   | ~~~~~

WARNING NG8113: IonAvatar is not used within the template of InfoPersonalPage [plugin angular-compiler]
```

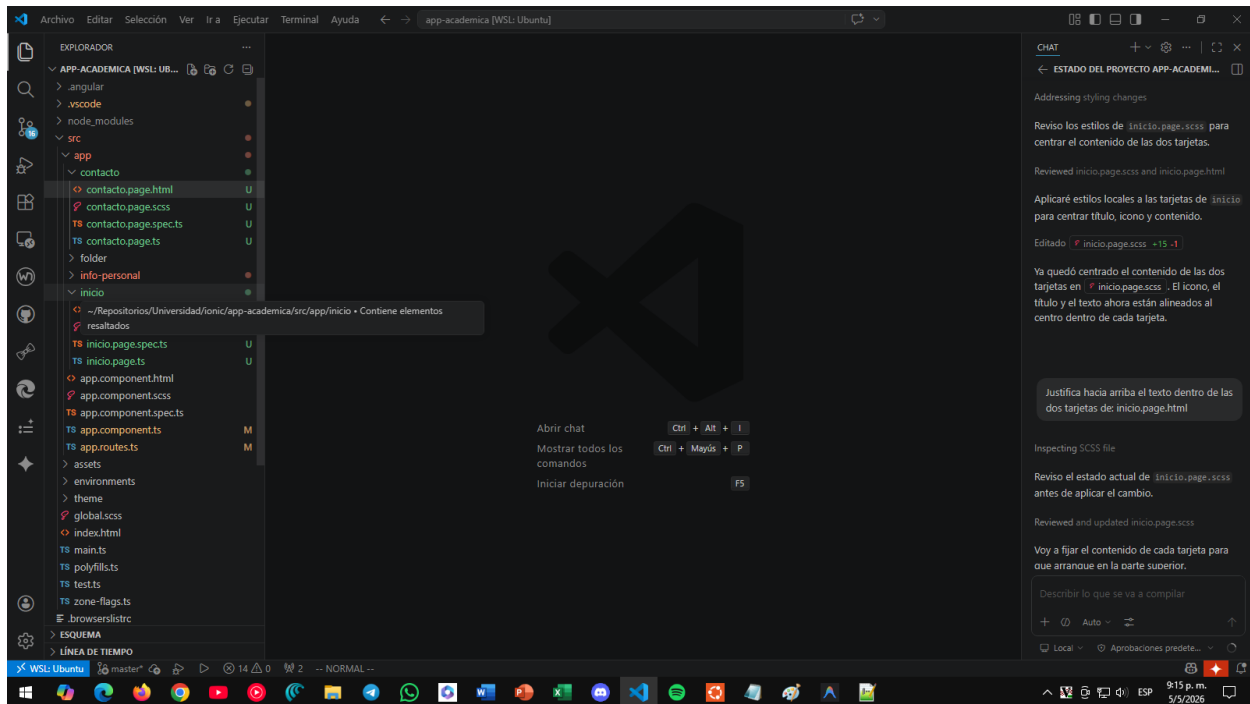


```
leonardo@DESKTOP-D4RBSC6: ~/Repositorios/Universidad/ionic/app-academica
[ng] main.js | main | 13.70 kB |
[ng] polyfills.js | polyfills | 143 bytes |
[ng] Initial total | 71.20 kB |
[ng] Lazy chunk files | Names | Raw size |
[ng] info-personal.page-OHAKUYKO.js | info-personal-page | 13.28 kB |
[ng] contacto.page-DI54VTQR.js | contacto-page | 10.36 kB |
[ng] inicio.page-FTJFXN4B.js | inicio-page | 8.98 kB |
[ng] Application bundle generation complete. [5.870 seconds] - 2026-05-08T00:37:49.861Z
[ng] ▲ [WARNING] NG8113: IonNote is not used within the template of AppComponent [plugin angular-compiler]
[ng] src/app/app.component.ts:12:109:
[ng] 12 | ...nt, IonList, IonListHeader, IonNote, IonMenuToggle, IonItem, Io...
[ng]                                     ~~~~~
[ng] ▲ [WARNING] NG8113: IonAvatar is not used within the template of InfoPersonalPage [plugin angular-compiler]
[ng] src/app/info-personal/info-personal.page.ts:18:217:
[ng] 18 | ..., IonIcon, IonLabel, IonGrid, IonRow, IonCol, IonAvatar, IonBadge]
[ng]                                     ~~~~~
[ng] NOTE: Raw file sizes do not reflect development server per-request transformations.
[INFO] Development server running!
Local: http://localhost:8100
Use Ctrl+C to quit this process
[ng] Local: http://localhost:8100/
[ng] press h + enter to show help
[INFO] Browser window opened to http://localhost:8100!
```

Renderizada la App:



Código en el editor:



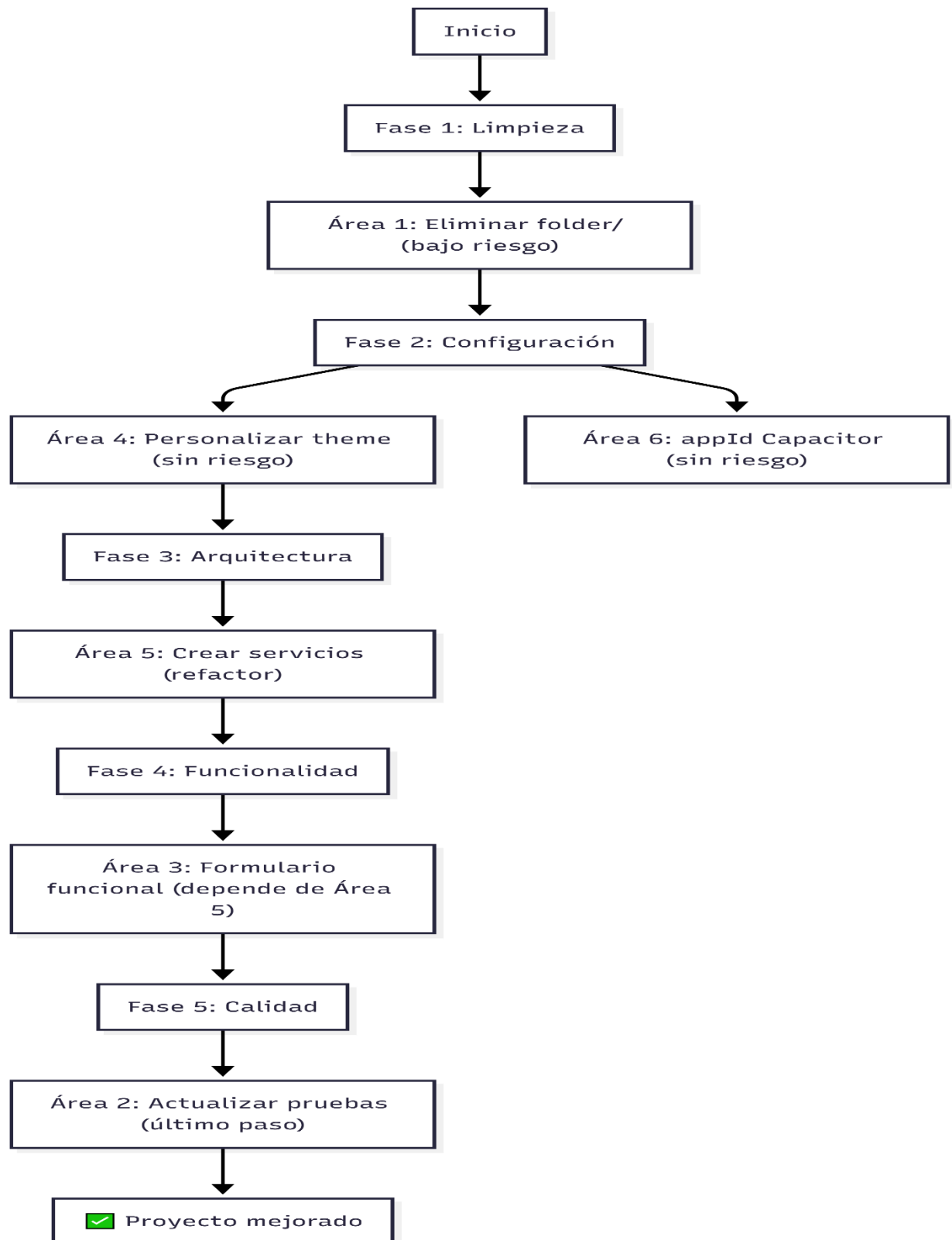
Mejoras Áreas Potenciales

Plan de Implementación: Áreas de Mejora Potencial

Resumen de las 6 Áreas a Mejorar

#	Área	Prioridad	Esfuerzo	Impacto
1	Página folder/ huérfana	Alta	Bajo	Limpieza
2	Pruebas desactualizadas (app.component.spec.ts)	Alta	Medio	Calidad
3	Formulario de contacto no funcional	Media	Alto	UX
4	variables.scss vacío (tema sin personalizar)	Media	Medio	UI/UX
5	Sin servicios (lógica en componentes)	Media	Alto	Arquitectura
6	appId genérico en Capacitor	Baja	Bajo	Configuración

Plan de Ejecución



Área 1: Eliminar Página folder/ Huérfana

Estado Actual

La carpeta src/app/folder/ contiene 4 archivos (page.ts, page.html, page.scss, page.spec.ts)

No está referenciada en app.routes.ts ni se usa en ninguna parte

Es código muerto (template por defecto de Ionic CLI)

Acciones a Realizar

#	Acción	Archivo(s)	Detalle
1.1	Eliminar carpeta src/app/folder/	folder.page.ts, folder.page.html, folder.page.scss, folder.page.spec.ts	Borrar directorio completo con sus 4 archivos
1.2	Verificar que no haya imports residuales	(buscar en todo el proyecto)	Asegurar que ningún otro archivo referencie FolderPage o './folder/folder.page'

Área 2: Corregir Pruebas Desactualizadas de app.component.spec.ts

Estado Actual

Las pruebas del AppComponent esperan 12 items del menú original de Ionic:

```

expect(menuItems.length).toEqual(12); // ← Espera 12, hay 3
expect(menuItems[0].textContent).toContain('Inbox'); // ← "Inicio"
expect(menuItems[1].textContent).toContain('Outbox'); // ← "Información Personal"
expect(menuItems[0].getAttribute('href')).toEqual('/folder/inbox'); // ← "/inicio"

```

Acciones a Realizar

#	Acción	Archivo	Detalle
2.1	Actualizar test should have menu labels	app.component.spec.ts	Corregir expectativas para 3 items del menú real
2.2	Actualizar test should have urls	app.component.spec.ts	Verificar href: /inicio, /info-personal, /contacto
2.3	Agregar test para redirección por defecto	app.routes.spec.ts (nuevo)	Crear prueba que verifique que " → /inicio
2.4	Ejecutar suite completa	Terminal	Verificar que todos los tests pasen

Área 3: Hacer Funcional el Formulario de Contacto

Estado Actual

El formulario en contacto.page.html tiene inputs visuales pero:

- Sin [(ngModel)] bindings ni FormGroup
- El botón "Enviar Mensaje" no tiene evento (click)
- Sin lógica de envío (ni siquiera simulación)
- Sin validaciones

Acciones a Realizar

#	Acción	Archivo	Detalle
3.1	Crear interface MensajeContacto	contacto/contacto.page.ts	Definir modelo con nombre, email, mensaje
3.2	Importar FormsModule y EmailComposer	contacto/contacto.page.ts	Para ngModel y envío nativo
3.3	Agregar propiedades reactivas al componente	contacto/contacto.page.ts	datosFormulario: MensajeContacto, método enviar()
3.4	Agregar [(ngModel)] a los inputs	contacto/contacto.page.html	Binding bidireccional en nombre, email y mensaje
3.5	Agregar evento (click)="enviar()" al botón	contacto/contacto.page.html	Vincular acción de envío
3.6	Implementar método enviar() con validación	contacto/contacto.page.ts	- Validar campos requeridos - Simular envío con Alert (demo) - Opcional: EmailComposer nativo
3.7	Agregar estilo de feedback visual	contacto/contacto.page.scss	Clases para validación/error
3.8	Actualizar ContactoPage import	contacto/contacto.page.ts	Agregar FormsModule a imports del standalone

Estructura de código resultante

```

Typescript
// contacto.page.ts
export interface MensajeContacto {
  nombre: string;
  email: string;
  mensaje: string;
}

@Component({
  // ...
  imports: [/* ... */, FormsModule]
})
export class ContactoPage {
  datosFormulario: MensajeContacto = { nombre: '', email: '', mensaje: '' };
  enviado = false;

  enviar() {
    if (!this.datosFormulario.nombre || !this.datosFormulario.email ||

```

Criterios de Aceptación

- Los campos del formulario capturan datos correctamente
- El botón "Enviar Mensaje" dispara una validación
- Si los campos están vacíos, muestra feedback visual de error
- Si los datos son válidos, muestra confirmación (Alert Toast)
- ng test pasa todas las pruebas

Área 4: Personalizar Tema (variables.scss)

Estado Actual

src/theme/variables.scss contiene solo un comentario. Se usan los colores por defecto de Ionic.

Acciones a Realizar

#	Acción	Archivo	Detalle
4.1	Definir paleta de colores de marca	src/theme/variables.scss	Colores primarios, secundarios, terciarios basados en identidad YI Technology / FinnovaVzla
4.2	Configurar modo oscuro personalizado	src/theme/variables.scss	Variables CSS para dark palette con overrides específicos
4.3	Agregar variables de espaciado y tipografía	src/theme/variables.scss	--ion-font-family, --ion-padding, etc.
4.4	Personalizar colores de componentes clave	src/theme/variables.scss	Toolbar, Cards, Badges, Botones
4.5	Verificar contraste visual	Navegador	Asegurar legibilidad en modo claro/oscuro

Propuesta de paleta

```

Scss
// src/theme/variables.scss

:root {
  /* === Paleta YI Technology / FinnovaVzla === */
  --ion-color-primary: #1a5276;          /* Azul profundo corporativo */
  --ion-color-primary-rgb: 26, 82, 118;
  --ion-color-primary-contrast: #ffffff;
  --ion-color-primary-shade: #15445f;
  --ion-color-primary-tint: #2d6b91;

  --ion-color-secondary: #2e86c1;        /* Azul medio tecnológico */
  --ion-color-secondary-rgb: 46, 134, 193;
  --ion-color-secondary-contrast: #ffffff;
  --ion-color-secondary-shade: #2471a3;
  --ion-color-secondary-tint: #3498db;

  --ion-color-tertiary: #1abc9c;         /* Verde fintech (prototipo) */

```

Área 5: Crear Servicios (Separar Lógica de Componentes)

Estado Actual

Toda la lógica está dentro de los componentes. No hay inyección de dependencias vía servicios.

Objetivo Arquitectónico

Separar responsabilidades siguiendo el principio de **Single Responsibility** de Angular. Esto permite:

- Mejor testabilidad (los servicios se prueban aisladamente)
- Reutilización de lógica entre componentes
- Mantenimiento más limpio

Acciones a Realizar

#	Acción	Archivo(s)	Detalle
5.1	Crear src/app/services/	Nueva carpeta	Directorio para servicios
5.2	Crear contacto.service.ts	src/app/services/contacto.service.ts	Clase inyectable @Injectable({ providedIn: 'root' }) con método enviarMensaje(data: MensajeContacto): Observable<boolean>
5.3	Refactorizar ContactoPage	contacto.page.ts	Inyectar ContactoService, delegar lógica de envío al servicio
5.4	Crear perfil.service.ts	src/app/services/perfil.service.ts	Método obtenerPerfil(): Observable<PerfilEstudiant e>
5.5	Refactorizar InfoPersonalPage	info-personal.page.ts	Inyectar PerfilService, obtener datos del perfil desde servicio

5.6	Crear navegacion.service.ts	src/app/services/navegacion.service.ts	Centralizar datos del menú (appPages) para posible reutilización
5.7	Actualizar AppComponent	app.component.ts	Consumir menú desde servicio
5.8	Crear barrel export index.ts	src/app/services/index.ts	Exportar todos los servicios

Estructura resultante de servicios

```

src/app/services/
├─ index.ts           # Barrel export
├─ contacto.service.ts # Lógica de formulario de contacto
├─ perfil.service.ts  # Datos del perfil profesional
└─ navegacion.service.ts # Configuración del menú de navegación

```

Ejemplo de contacto.service.ts

```

Typescript
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { MensajeContacto } from '../contacto/contacto.page';

@Injectable({
  providedIn: 'root'
})
export class ContactoService {
  enviarMensaje(data: MensajeContacto): Observable<boolean> {
    // TODO: Integrar con API real o EmailComposer de Capacitor
    console.log('[ContactoService] Mensaje recibido:', data);
    return of(true); // Simulación exitosa
  }
}

```



Main Config

Área 6: Personalizar appId de Capacitor

Estado Actual

```

Typescript
// capacitor.config.ts
const config: CapacitorConfig = {
  appId: 'io.ionic.starter',    // ← Genérico
  appName: 'app-academica',    // ← Nombre temporal
  webDir: 'www'
};
  
```

Acciones a Realizar

#	Acción	Archivo	Detalle
6.1	Cambiar appId	capacitor.config.ts	Usar dominio real: com.yitechnology.appacademica o com.finnovavzla.portfolio
6.2	Cambiar appName	capacitor.config.ts	Nombre visible para el usuario: "App Académica" o "FinnovaVzla Portfolio"
6.3	Agregar metadatos opcionales	capacitor.config.ts	server.url, server.cleartext si aplica
6.4	Sincronizar Capacitor	Terminal	npx cap sync para regenerar config nativa

Resultado esperado

```
Typescript
// capacitor.config.ts
import type { CapacitorConfig } from '@capacitor/cli';

const config: CapacitorConfig = {
  appId: 'com.finnovavzla.appacademica',
  appName: 'FinnovaVzla Portfolio',
  webDir: 'www',
  // Opcional: para debug remoto
  // server: {
  //   url: 'http://localhost:8100',
  //   cleartext: true
  // }
};

export default config;
```

Código fuente app académica: Dev Portafolio

<https://github.com/l3og2/app-academica.git>

Conclusiones

El desarrollo de esta aplicación móvil demostró de manera práctica cómo las herramientas modernas de desarrollo web pueden adaptarse para crear experiencias nativas en dispositivos móviles. Al finalizar el proyecto, se logró construir una interfaz fluida, responsiva y orientada al usuario, cumpliendo con la estructura requerida de tres menús de navegación independientes.

Desde el punto de vista técnico, la implementación de la arquitectura de componentes Standalone de Angular representó un avance significativo en la optimización del proyecto. Esta decisión permitió aplicar la "carga perezosa" (Lazy Loading), asegurando que el dispositivo del usuario solo procese el código de la pantalla que está visitando en ese instante, ahorrando memoria y batería. Además, el uso de componentes visuales preconstruidos de Ionic, como las grillas, tarjetas y formularios interactivos, garantizó un diseño profesional e intuitivo sin necesidad de escribir código de estilos (CSS) redundante.

En conclusión, este ejercicio académico consolida los conocimientos de programación Front-End y diseño de interfaces. La aplicación desarrollada no solo es totalmente funcional en su estado actual, sino que posee una estructura de código limpia y escalable, dejándola preparada para integrarse en el futuro con bases de datos reales o interfaces de programación (APIs) en el lado del servidor.

Referencias

IDLE Visual Estudio Code – mayo 2026.

<https://code.visualstudio.com/>

Gems DevMaster por Leonardo González – febrero 2026

<https://gemini.google.com/app>

Angular Team. (2024). *Standalone components*. Google. Recuperado el 7 de mayo de 2026, de <https://angular.dev/guide/components/standalone>

Angular Team. (2024). *Built-in control flow*. Google. Recuperado el 7 de mayo de 2026, de <https://angular.dev/guide/templates/control-flow>

Ionic Framework. (2024). *UI Components: Build Awesome Mobile Apps*. Drifty Co.

Recuperado el 7 de mayo de 2026, de <https://ionicframework.com/docs/components>

Ionic Framework. (2024). *Angular Navigation & Routing*. Drifty Co. Recuperado el 7 de mayo de 2026, de <https://ionicframework.com/docs/angular/navigation>

Mozilla Developer Network (MDN). (2023). *Estructurando la web con HTML*. Mozilla

Foundation. Recuperado el 7 de mayo de 2026, de

https://developer.mozilla.org/es/docs/Learn/HTML/Introduction_to_HTML